

Computer Science Portfolio

Project Report

Anish Rana Magar

Gisma University of Applied Sciences

B.Sc. Computer Science

Portfolio URL: <https://anishranamagar.github.io/protfolio>

Repository URL: <https://github.com/anishranamagar/protfolio>

Submission date: June 26, 2026

Contents

1	Introduction	2
2	Structure of the Portfolio	2
2.1	Repository Layout	2
2.2	Website Layout	3
3	Design Decisions	3
3.1	Single-Page Architecture	3
3.2	Dark Theme with a Green Accent	3
3.3	Terminal Hero as the Signature Element	4
3.4	Plain HTML and CSS (No Framework)	4
3.5	Responsive Design	4
3.6	Scroll-Reveal Animation	4
3.7	LaTeX for the CV	5
4	Tools and Technologies	5
4.1	GitHub Pages	5
4.2	Version Control Workflow	5
5	Portfolio Content	6
5.1	About Section	6
5.2	Skills Section	6
5.3	Projects Section	6
5.4	CV	6
6	Reflection: Strengths, Weaknesses, and Future Work	7
6.1	Strengths	7
6.2	Weaknesses	7
6.3	Planned Future Improvements	7
7	Conclusion	8

Introduction

This report describes the design, implementation, and content of a computer science portfolio developed as part of a working student application. The portfolio serves two audiences simultaneously: potential employers who want a quick overview of technical competence, and technical reviewers who wish to inspect source code, commit history, and documentation quality directly.

The portfolio is hosted as a static website on GitHub Pages and is accessible at:

<https://anishranamagar.github.io/protfolio>

The source code and all supplementary materials are version-controlled in a public GitHub repository at:

<https://github.com/anishranamagar/protfolio>

The remainder of this report is structured as follows. Section 2 describes the repository and website layout. Section 3 justifies the design decisions taken. Section 4 documents the tools and technologies used. Section 5 explains the portfolio content in detail. Section 6 reflects on strengths, weaknesses, and future improvements.

Structure of the Portfolio

Repository Layout

The GitHub repository follows a flat, clearly named directory structure to make navigation straightforward for any visitor:

Listing 1: Repository directory tree

```
1 anishranamagar.github.io/protfolio/
2     index.html           # Single-page portfolio website
3     assets/
4         css/
5             style.css    # All visual styles
6     cv/
7         cv.tex           # LaTeX CV source
8         cv.pdf           # Compiled PDF (downloadable)
9     projects/
10         data-analysis/  # Python grade analyser project
11         library-system/ # Java library management
12     project
13         db-design/      # SQL database schema project
14     README.md           # Repository overview and
```

navigation guide

Each project subdirectory contains its source files, a `README.md` explaining the project's purpose and how to run it, and (where applicable) sample output. This mirrors industry conventions for open-source projects and makes it easy for reviewers to evaluate individual pieces of work without loading the website.

Website Layout

The website is a single-page application divided into five clearly labelled sections:

1. **Hero** — an interactive terminal widget displaying key information and two call-to-action buttons.
2. **About** — a short biography and four summary statistics cards.
3. **Skills** — an annotated skills grid with proficiency bars and tag clouds for tools and concepts.
4. **Projects** — three project cards linking to the repository subdirectories.
5. **Contact** — email, GitHub, and LinkedIn links.

A sticky navigation bar provides direct access to each section at all times.

Design Decisions

Single-Page Architecture

A multi-page website would fragment the visitor's attention across multiple HTTP requests and page loads. Since the portfolio's content fits comfortably on one page, a single `index.html` reduces load time, simplifies navigation, and allows the visitor to scroll through the entire portfolio in one uninterrupted reading session. The smooth-scroll JavaScript and the sticky navigation bar together make the single-page experience as navigable as a multi-page site.

Dark Theme with a Green Accent

A dark background (`#0d0f14`) with a bright green accent (`#00e5a0`) was chosen deliberately for two reasons. First, it is immediately recognisable as a programming aesthetic — terminal emulators and code editors have conditioned developers to associate this palette with technical competence. Second, the high contrast ratio between the accent and the background ($> 7:1$ measured by WCAG 2.1) ensures readability for users with colour-vision deficiencies.

Terminal Hero as the Signature Element

Rather than a generic photograph or abstract illustration, the hero section features a simulated terminal window. This is the single “signature element” of the design: it is specific to the subject (a computer scientist), communicates key information (name, role, skills) in a format familiar to developers, and creates an immediate impression that the site was built by someone who understands the field. The blinking cursor animation reinforces this impression without adding unnecessary noise.

Plain HTML and CSS (No Framework)

The site is written in plain HTML5 and CSS3, without relying on frameworks such as Bootstrap or Tailwind. This decision was made for three reasons:

- **Demonstrates fundamentals.** A hiring manager reviewing the source code sees that the author understands CSS from first principles rather than relying on a dependency.
- **No build step.** A plain HTML/CSS site can be deployed to GitHub Pages by simply pushing files to the repository. No Node.js toolchain, no compilation, no `node_modules` directory.
- **Performance.** With no JavaScript framework and no external CSS library, the total page weight is minimal, resulting in fast load times.

The only external dependency is two Google Fonts families (*IBM Plex Mono* and *Inter*), loaded via a single CDN request.

Responsive Design

The layout uses CSS Grid with `auto-fit` / `minmax` patterns and a single `@media` breakpoint at 768px width. Below this breakpoint the two-column grids collapse to a single column, the desktop navigation is replaced by a hamburger menu, and the hero sections reflow to a vertical stack. This ensures the portfolio is fully usable on mobile devices, which is important as recruiters commonly review portfolios on smartphones.

Scroll-Reveal Animation

A lightweight `IntersectionObserver` script applies a fade-in-and-rise animation to sections as they enter the viewport. The effect is subtle (20px vertical translation, 0.6s duration) and is disabled entirely for users who have enabled the “Reduce Motion” accessibility preference (`prefers-reduced-motion`).

LaTeX for the CV

The curriculum vitae is typeset in $\text{\LaTeX}$ rather than produced with a word processor or an online CV builder. This choice signals familiarity with academic and technical writing tools common in computer science, produces a PDF with precise typography, and keeps the source file under version control alongside the rest of the portfolio.

Tools and Technologies

Table 1 summarises the tools and technologies used to develop and deploy the portfolio.

Table 1: Tools and technologies used

Tool / Technology	Version / Service	Purpose
HTML5	Standard	Page markup and semantic structure
CSS3	Standard	All visual styling and layout
JavaScript (ES6+)	Vanilla	Navigation scroll, IntersectionObserver reveal
Google Fonts	CDN	IBM Plex Mono (monospace), Inter (body)
GitHub	git 2.x	Version control and repository hosting
GitHub Pages	Free tier	Static site deployment and hosting
$\text{\LaTeX}$	TeX Live 2024	CV and report typesetting
Overleaf	Online editor	Cloud-based $\text{\LaTeX}$ editing and compilation
VS Code	1.89	HTML/CSS/JavaScript editing

GitHub Pages

GitHub Pages serves static files directly from a repository, making it the natural choice for hosting a portfolio alongside its source code. The setup requires only that the repository be named `username.github.io` and that GitHub Pages be enabled in the repository settings. No server configuration, no cost, and the deployment is automatic on every `git push`.

Version Control Workflow

All changes to the portfolio are committed to the `main` branch with descriptive commit messages following the Conventional Commits specification (e.g. `feat: add projects section`, `fix: correct mobile nav overflow`). This makes the development history readable and demonstrates awareness of professional version-control practices.

Portfolio Content

About Section

A short biography introduces the visitor to the author's background, academic focus, and career goal. Four "stat cards" (years of programming, number of projects, languages spoken, percentage of documented commits) provide scannable key facts in a format that is immediately digestible.

Skills Section

Skills are divided into three groups: programming languages (with approximate proficiency percentages), tools and platforms (presented as a tag cloud), and computer science concepts (also a tag cloud). The proficiency percentages are self-assessed and intentionally conservative to avoid overstating ability.

Projects Section

Three projects are showcased:

Student Grade Analyser (Python). A data-processing script demonstrating file I/O, pandas, and matplotlib. Selected to show competence in the language most commonly requested in working-student job descriptions.

Library Management System (Java). A console application demonstrating OOP principles and unit testing. Selected to show proficiency in a statically typed language and familiarity with software engineering best practices.

University Database Schema (SQL). A database design exercise demonstrating ER modelling, normalisation, and complex query writing. Selected to show understanding of relational databases, a skill required in most software development roles.

Each project is hosted in its own subdirectory of the repository and includes a `README.md` with a description, setup instructions, and example output.

CV

The curriculum vitae is compiled from `cv/cv.tex` and hosted as `cv/cv.pdf`. A "Download CV" button in both the hero section and the dedicated CV section links to this file. Keeping the source `.tex` file in the repository allows version-controlled updates and demonstrates $\text{\LaTeX}$ proficiency.

Reflection: Strengths, Weaknesses, and Future Work

Strengths

- **Technical transparency.** The portfolio is itself a technical artefact: the source code is public, well-structured, and commented. A reviewer can inspect the actual HTML, CSS, and JavaScript rather than only reading about them.
- **Accessibility.** The site respects `prefers-reduced-motion`, uses semantic HTML5 elements (`<nav>`, `<header>`, `<section>`, `<article>`, `<footer>`), and maintains sufficient colour contrast for WCAG 2.1 AA compliance.
- **Performance.** No JavaScript framework or CSS library means the page loads in under one second on a standard connection.
- **Version control.** The entire development history is visible in the Git log, demonstrating disciplined, incremental development.

Weaknesses

- **Limited project depth.** The three showcased projects are academic exercises rather than deployed products. A hiring manager looking for evidence of collaborative, production-scale work will not find it here.
- **No back-end or deployment experience shown.** The portfolio is a static site; it does not demonstrate knowledge of servers, APIs, or cloud infrastructure.
- **No automated tests or CI/CD pipeline.** A professional repository would include a continuous integration workflow (e.g. GitHub Actions) that validates the HTML, runs a linter, and checks accessibility automatically.

Planned Future Improvements

1. **Add a real-world collaborative project.** Contribute meaningfully to an open-source project and link to the merged pull request.
2. **Introduce a GitHub Actions CI workflow.** Automate HTML validation (`html-validate`), CSS linting (`stylelint`), and accessibility checking (`axe-core`) on every push.
3. **Add a blog section.** Short technical articles (e.g. write-ups of algorithm implementations) would demonstrate communication skills and make the portfolio a living document rather than a static snapshot.
4. **Dark/light mode toggle.** Implement CSS custom properties toggled by a button, storing the preference in `localStorage`, to demonstrate JavaScript skills and improve usability.

5. **More projects across different domains.** Add a machine-learning notebook (Python/scikit-learn) and a small web API (Python/Flask or Node.js/Express) to show breadth.

Conclusion

This report has described the design rationale, technical implementation, and content of a computer science portfolio hosted on GitHub Pages. The key design principles — semantic HTML, plain CSS, terminal aesthetic, and full version-control transparency — were chosen to demonstrate both technical competence and professional software development habits to a technical hiring audience.

The portfolio is a starting point rather than a finished product. The reflection in Section 6 identifies concrete next steps that will deepen and broaden the showcased skills as the degree programme progresses.